Mauricio de Souza Coutinho

Exercício 1:

Liste produtos com preço acima da média geral:

```
64 •  SELECT *
65    FROM produtos
66    WHERE preco > (SELECT AVG(preco) FROM produtos);
```

Result Grid | Filter Rows: | Edit:

id_produto	nome	categoria	preco
103	Notebook	ELETRONICOS	3500.00
NULL	NULL	NULL	NULL

Localize clientes com ao menos um pedido pago usando IN:

```
68 •  SELECT *
69    FROM clientes
70    WHERE id_cliente IN (
71        SELECT id_cliente
72        FROM pedidos
73        WHERE status = 'PAGO'
74    );
```

Result Grid | Filter Rows:

id_cliente	nome	email
1	Ana Silva	ana@email.com
2	Bruno Souza	bruno@email.com
NULL	NULL	NULL

Reescreva usando EXISTS:

```
76 •  SELECT c.*
77    FROM clientes c
78    WHERE EXISTS(
79        SELECT 1
80        FROM pedidos p
81        WHERE p.id_cliente = c.id_cliente
82        AND p.status = 'PAGO'
83        )
```

Result Grid | Filter Rows: | Edit:

id_cliente	nome	email
1	Ana Silva	ana@email.com
2	Bruno Souza	bruno@email.com
NULL	NULL	NULL

Encontre clientes sem pedidos usando NOT EXISTS:

```
85  SELECT c.*
86  FROM clientes c
87  WHERE NOT EXISTS(
88      SELECT 1
89      FROM pedidos p
90      WHERE p.id_cliente = c.id_cliente
91  );
```

Result Grid | Filter Rows: | Edit:

id_cliente	nome	email
4	Diego Lima	diego@email.com
NULL	NULL	NULL

Compare ANY e ALL com os preços da categoria livro:

```
93  -- Preco maior que QUALQUER livro (maior que o menor livro)
94  SELECT *
95  FROM produtos
96  WHERE preco > ANY (SELECT preco FROM produtos WHERE categoria = 'LIVRO');
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Ce

id_produto	nome	categoria	preco
101	Livro de SQL Avançado	LIVRO	120.00
103	Notebook	ELETRONICOS	3500.00
104	Mouse Gamer	ELETRONICOS	150.00
105	Livro de Python	LIVRO	90.00
NULL	NULL	NULL	NULL

```
98  SELECT *
99  FROM produtos
100 WHERE preco > ALL (SELECT preco FROM produtos WHERE categoria = 'LIVRO');
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Ce

id_produto	nome	categoria	preco
103	Notebook	ELETRONICOS	3500.00
104	Mouse Gamer	ELETRONICOS	150.00
NULL	NULL	NULL	NULL

Exercício 2:

Liste produtos acima da média da própria categoria:

```
104 SELECT p1.*
105 FROM produtos p1
106 WHERE p1.preco >(
107     SELECT AVG (p2.preco)
108     FROM produtos p2
109     WHERE p2.categoria = p1.categoria
110 );
```

Result Grid | Filter Rows: | Edit:

id_produto	nome	categoria	preco
101	Livro de SQL Avançado	LIVRO	120.00
103	Notebook	ELETRONICOS	3500.00
NULL	NULL	NULL	NULL

Recupere o pedido mais recente de cada cliente:

```sql
SELECT p1.*
FROM pedidos p1
WHERE p1.data_pedido = (
    SELECT MAX(p2.data_pedido)
    FROM pedidos p2
    WHERE p2.id_cliente = p1.id_cliente
);
```

Result Grid | Filter Rows: | Edit:

id_pedido	id_cliente	data_pedido	status
1002	1	2023-10-15	PAGO
1003	2	2023-09-20	PAGO
1004	3	2023-10-05	CANCELADO
NULL	NULL	NULL	NULL

Reescreva uma correlação usando JOIN com agregação prévia:

```sql
SELECT p.*
FROM produtos p
JOIN (
    SELECT categoria , AVG(preco) AS media_preco
    FROM produtos
    GROUP BY categoria
) media_cat ON p.categoria = media_cat.categoria
WHERE p.preco > media_cat.media_preco;
```

Result Grid | Filter Rows: | Export: | Wrap Cell

id_produto	nome	categoria	preco
101	Livro de SQL Avançado	LIVRO	120.00
103	Notebook	ELETRONICOS	3500.00

Crie uma CTE para calcular faturamento por produto:

```sql
WITH FaturamentoProduto AS (
    SELECT
        id_produto,
        SUM(quantidade * preco_unitario) AS faturamento_total
    FROM itens_pedido
    GROUP BY id_produto
)
SELECT p.nome, fp.faturamento_total
FROM FaturamentoProduto fp
JOIN produtos p ON p.id_produto = fp.id_produto;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

nome	faturamento_total
Livro de SQL Avançado	120.00
Livro de Banco de Dados	80.00
Notebook	3500.00
Mouse Gamer	300.00
Livro de Python	90.00

Compare clareza , cardinalidade e provável custo das versões:

5. Comparação de Clareza, Cardinalidade e Custo Provável:

Clareza: A solução com CTE e JOIN com Agregação Prévia é mais modular e legível que subconsultas correlacionadas.

Cardinalidade: Subconsultas correlacionadas executam linha a linha (comportamento O(N x M) se não indexado), enquanto JOIN com agregação prévia e CTE pré-agrupam os dados (O(N + M)).

Custo Provável: Em volumes de dados grandes, o JOIN com subconsulta/CTE costuma ter menor custo computacional (evita *scans* repetidos da tabela interna).

Exercício Integrador:

Duas subconsultas escalares e duas de múltiplas linhas:

Escalar:

```
140     -- Escalares (retornam um unico valor)
141     SELECT nome , preco ,
142         (SELECT AVG (preco) FROM produtos) AS media_geral ,
143         (preco - (SELECT AVG (preco) FROM produtos)) AS diferenca_da_media
144     FROM produtos;
145
146
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

nome	preco	media_geral	diferenca_da_media
Livro de SQL Avançado	120.00	788.000000	-668.000000
Livro de Banco de Dados	80.00	788.000000	-708.000000
Notebook	3500.00	788.000000	2712.000000
Mouse Gamer	150.00	788.000000	-638.000000
Livro de Python	90.00	788.000000	-698.000000

Múltiplas linhas:

```
146     SELECT *
147     FROM produtos
148     WHERE id_produto IN (
149         SELECT id_produto
150         FROM itens_pedido
151         WHERE id_pedido IN (SELECT id_pedido FROM pedidos WHERE status = 'PAGO')
152     );
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Co

id_produto	nome	categoria	preco
101	Livro de SQL Avançado	LIVRO	120.00
102	Livro de Banco de Dados	LIVRO	80.00
103	Notebook	ELETRONICOS	3500.00
104	Mouse Gamer	ELETRONICOS	150.00
NULL	NULL	NULL	NULL

Uma solução com EXISTS e outra com NOT EXISTS:

EXISTS:

```sql
SELECT p.*
FROM produtos p
WHERE EXISTS (
    SELECT 1 FROM itens_pedido ip WHERE ip.id_produto = p.id_produto
);
```

Result Grid | Filter Rows: | Edit: | Export/Import:

id_produto	nome	categoria	preco
101	Livro de SQL Avançado	LIVRO	120.00
102	Livro de Banco de Dados	LIVRO	80.00
103	Notebook	ELETRONICOS	3500.00
104	Mouse Gamer	ELETRONICOS	150.00
105	Livro de Python	LIVRO	90.00
NULL	NULL	NULL	NULL

NOT EXISTS:

```sql
SELECT p.*
FROM produtos p
WHERE NOT EXISTS (
    SELECT 1 FROM itens_pedido ip WHERE ip.id_produto = p.id_produto
);
```

Result Grid | Filter Rows: | Edit: | Export/Import:

id_produto	nome	categoria	preco
NULL	NULL	NULL	NULL

Uma subconsulta correlacionada e sua reescrita:

```sql
-- Correlacionada
SELECT c.nome
FROM clientes c
WHERE (
    SELECT COUNT(*)
    FROM pedidos p
    WHERE p.id_cliente = c.id_cliente
) > 1;
```

Result Grid | Filter Rows: | Export:

nome
Ana Silva

```sql
-- Reescrita (Usando JOIN + GROUP BY)
SELECT c.nome
FROM clientes c
JOIN pedidos p ON c.id_cliente = p.id_cliente
GROUP BY c.id_cliente, c.nome
HAVING COUNT(p.id_pedido) > 1;
```

Result Grid | Filter Rows: | Export: | Wra

nome
Ana Silva

Duas CTEs encadeadas para produzir um indicador:

```sql
WITH VendasPorCliente AS (
    SELECT
        p.id_cliente,
        SUM(ip.quantidade * ip.preco_unitario) AS total_gasto
    FROM pedidos p
    JOIN itens_pedido ip ON p.id_pedido = ip.id_pedido
    WHERE p.status = 'PAGO'
    GROUP BY p.id_cliente
),

MediaGastoGeral AS (
SELECT
    vc.id_cliente,
    vc.total_gasto,
    m.avg_gasto,
    CASE
        WHEN vc.total_gasto >= m.avg_gasto THEN 'Cliente VIP'
        ELSE 'Cliente Padrão'
    END AS classificacao
FROM VendasPorCliente vc, MediaGastoGeral m;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id_cliente	total_gasto	avg_gasto	classificacao
1	3920.00	2000.000000	Cliente VIP
2	80.00	2000.000000	Cliente Padrão

Uma view que simplifique uma consulta recorrente:

```sql
CREATE VIEW vw_resumo_pedidos AS
SELECT
    p.id_pedido,
    c.nome AS nome_cliente,
    p.data_pedido,
    p.status,
    COALESCE(SUM(ip.quantidade * ip.preco_unitario), 0) AS valor_total
FROM pedidos p
JOIN clientes c ON p.id_cliente = c.id_cliente
LEFT JOIN itens_pedido ip ON p.id_pedido = ip.id_pedido
GROUP BY p.id_pedido, c.nome, p.data_pedido, p.status;

SELECT *
FROM vw_resumo_pedidos;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id_pedido	nome_cliente	data_pedido	status	valor_total
1001	Ana Silva	2023-10-01	PAGO	420.00
1002	Ana Silva	2023-10-15	PAGO	3500.00
1003	Bruno Souza	2023-09-20	PAGO	80.00
1004	Carla Dias	2023-10-05	CANCELADO	90.00

Uma view que restrinja colunas expostas , com justificativa de privilégios:

```sql
SELECT *
FROM vw_resumo_pedidos;

CREATE VIEW vw_catalogo_publico AS
SELECT
    id_produto,
    nome,
    categoria,
    preco
FROM produtos;

SELECT *
FROM vw_catalogo_publico
WHERE categoria = 'LIVRO';
/*
JUSTIFICATIVA DE PRIVILÉGIOS DE ACESSO:
A View 'vw_catalogo_publico' expõe apenas as colunas informativas de produtos,
ocultando dados sensíveis ou de uso estritamente interno (como tabelas de custo,
margem de lucro, dados cadastrais de clientes ou históricos financeiros dos pedidos).
Em um ambiente corporativo, atribui-se o privilégio SELECT nesta view para papéis (roles)
de usuários finais ou integrações públicas, garantindo o princípio do menor privilégio (Least Privilege).
*/
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id_produto	nome	categoria	preco
101	Livro de SQL Avançado	LIVRO	120.00
102	Livro de Banco de Dados	LIVRO	80.00
105	Livro de Python	LIVRO	90.00